

Storyline สไลด์: Retail Product Recommendations

ที่มาของตัวเลขทุกตัว (คำนวณยังไง ทำไมเลือกค่านี) อยู่ในภาคผนวกท้ายเอกสาร

แนวเรื่อง: เล่าเป็นเรื่องของ "เจ้าของร้านของชำที่ไม่ว่าลูกค้าซื้ออะไรคู่กัน" แล้วให้อัลกอริทึมแต่ละตัวเข้ามาไขปัญหาทีละขั้น ท้ายเรื่องคือคำตอบที่ร้านเอาไปใช้ได้จริง

โครงเรื่อง 4 องก์ รวม 16 สไลด์ ประมาณ 13-15 นาที

องก์	เล่าเรื่องอะไร	สไลด์	เวลา
1. ปัญหา	ร้านมีข้อมูลเยอะแต่ไม่รู้ว่าจะใช้ยังไง	1-4	2.5 นาที
2. อัลกอริทึม (ส่วนหลัก)	AI หาความสัมพันธ์ ทีละขั้น	5-10	6 นาที
3. พิสูจน์	กฎที่ได้เชื่อได้จริงไหม	11-12	2 นาที
4. Output	ร้านได้อะไรกลับไป	13-16	3.5 นาที

ในแต่ละสไลด์มี

- ภาพ: กราฟจาก Colab (เลข 01-12 ใน slides/) หรือภาพจาก GPT (ดู image_prompts.md)
- ต้องพูด: ประเด็นที่ห้ามขาด
- ประโยคส่งต่อ: พูดปิดท้ายสไลด์ เพื่อพาเข้าสไลด์ถัดไปให้เป็นเรื่องเดียวกัน

องก์ 1: ปัญหา

สไลด์ 1: ปก

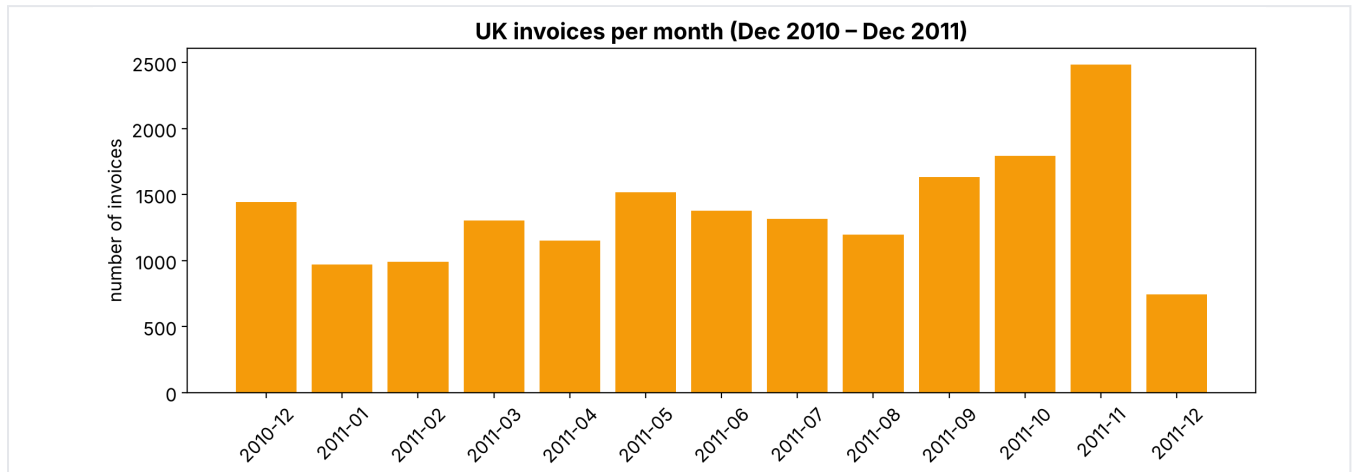
- ภาพ: ภาพ GPT prompt 1
- ต้องพูด:
 - ชื่อโปรเจกต์
 - ชื่อสมาชิก
 - ประโยคเดียวว่าทำอะไร: "ใช้ AI หาว่าสินค้าอะไรขายคู่กัน แล้วนำไปจัดโปรและจัดวางสินค้า"

สไลด์ 2: เปิดเรื่องด้วยคำถาม

- ภาพ: ภาพ GPT prompt 2 (เจ้าของร้านงกกับใบเสร็จ)
- ต้องพูด:
 - ชวนคนฟังนึกถึงร้านสะดวกซื้อที่วางขนมไว้ข้างน้ำอัดลม หรือหน้า "Frequently bought together" ใน Shopee
 - ร้านใหญ่รู้ว่าอะไรขายคู่กันเพราะมีทีม data แต่ร้านเล็กไม่รู้ ได้แต่เดา
 - โจทย์ของเรา: ร้านมีใบเสร็จเป็นแสนแถว จะหาจากใบเสร็จได้ไหมว่าอะไรขายคู่กัน
- ประโยคส่งต่อ: "เราเลยไปหาข้อมูลจากร้านจริงมาลองครับ"

สไลด์ 3: รู้จักข้อมูล

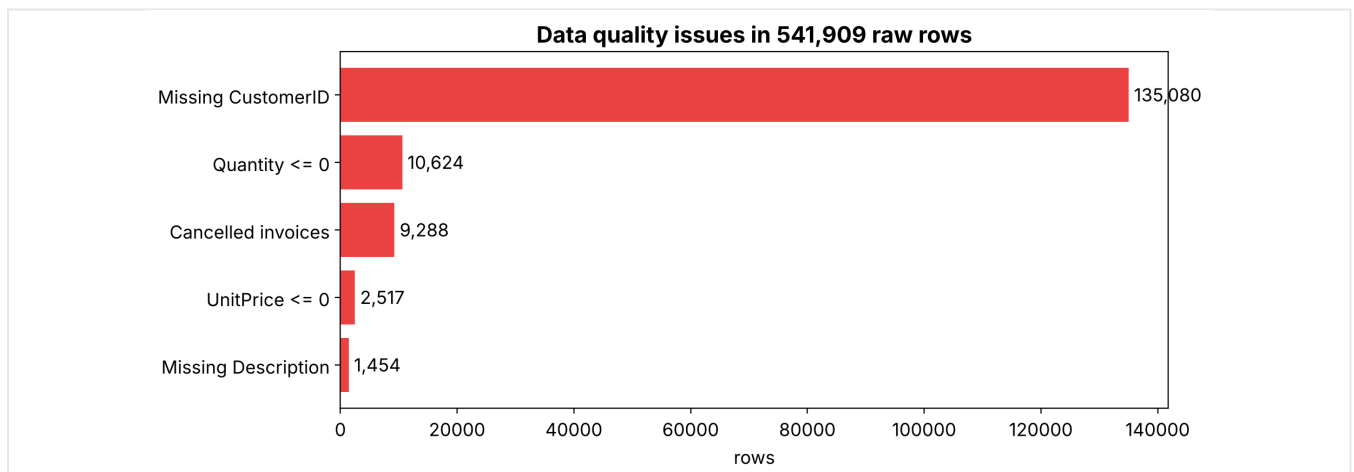
- ภาพ: กราฟ 02 (ยอดใบเสร็จรายเดือน), 03 (สินค้าขายดี)



- ต้องพูด:
 - ร้านขายของขวัญออนไลน์ในอังกฤษ ข้อมูล ธ.ค. 2010 – ธ.ค. 2011
 - ใบเสร็จจริง 541,909 แถว เลือกใช้เฉพาะลูกค้าในอังกฤษ
 - ยอดพุ่งช่วง ก.ย.-พ.ย. เพราะเป็นช่วงซื้อของขวัญคริสต์มาส (จำจุดนี้ไว้ เดี่ยวจะกลับมาเล่าในองก์ 3)
- ประโยคส่งต่อ: "แต่ข้อมูลจริงไม่สะอาดครับ"

สไลด์ 4: ทำความสะอาด และแปลงเป็นตะกร้า

- ภาพ: กราฟ 01 (ปัญหาในข้อมูล), 04 (จำนวนสินค้าต่อใบเสร็จ)



- ต้องพูด:
 - สิ่งที่ต้องทิ้ง:
 - ใบเสร็จยกเลิก (เลขขึ้นต้นด้วย C)
 - จำนวนหรือราคาติดลบ
 - ค่าส่งที่ไม่ใช่สินค้า (POSTAGE)
 - เหลือ 527,809 แถว
 - หัวใจของสไลด์นี้: แปลงจาก "แถว" เป็น "ตะกร้า" ด้วย TransactionEncoder
 - ได้ตาราง 17,917 ใบเสร็จ × 3,992 สินค้า แต่ละช่องคือ True หรือ False ว่าใบเสร็จนั้นมีสินค้าตัวนั้นไหม
 - ใบเสร็จหนึ่งใบมีสินค้าเฉลี่ย 26 ชิ้น

- ประโยคส่งต่อ: "พอได้ตะกร้าแล้ว คำถามคือจะหาคู่สินค้าจากราง $18,000 \times 4,000$ ได้อย่างไร"

องค์ 2: อัลกอริทึม (ส่วนที่ควรใช้เวลามากที่สุด)

สไลด์ 5: ทำไมนับตรงๆ ไม่ได้

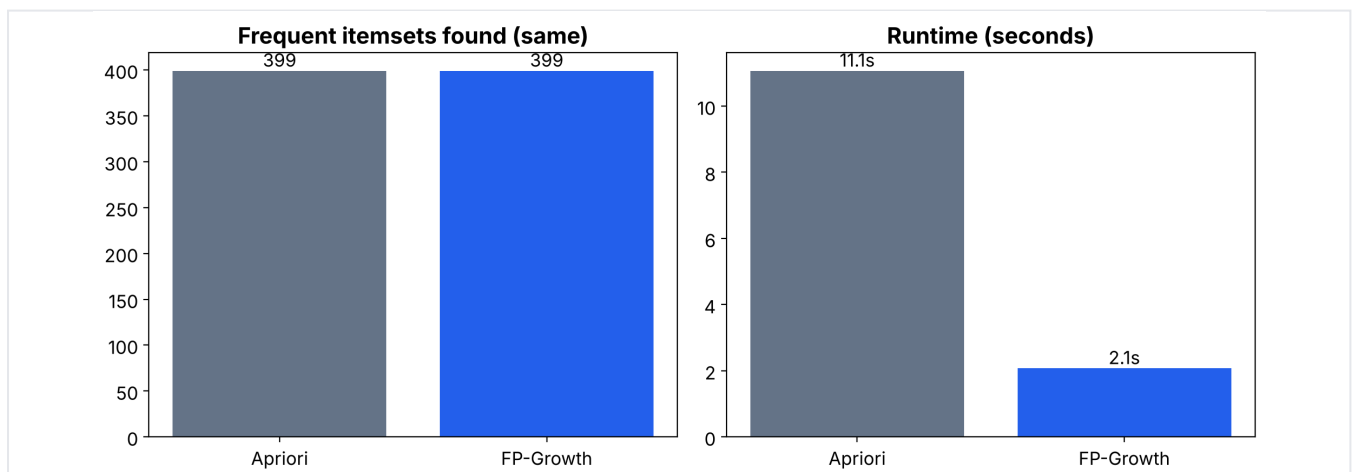
- ภาพ: วาดเอง ตัวเลขใหญ่ๆ บนสไลด์
- ต้องพูด:
 - สินค้า 3,992 ตัว ถ้าจับคู่ทุกแบบจะได้ประมาณ 8 ล้านคู่ ถ้าเป็นชุดละ 3 ชิ้นจะได้มากกว่า 1 หมื่นล้านชุด
 - นับทั้งหมดในทุกใบเสร็จไม่ไหว เราต้องมีวิธีตัดตัวเลือกทิ้ง
- ประโยคส่งต่อ: "อัลกอริทึมตัวแรกที่แก้ปัญหานี้ชื่อ Apriori ครับ"

สไลด์ 6: Apriori ทำงานยังไง

- ภาพ: วาดแผนภาพต้นไม้ 3 ชั้น (สินค้าเดี่ยว → คู่ → ชุด 3 ชิ้น) และกากบาทกิ่งที่ถูกตัดทิ้ง
- ต้องพูด:
 - กฎหลัก (Apriori principle): "ถ้าสินค้าตัวเดียวขายไม่บ่อย คู่ที่มีมันอยู่ก็ขายไม่บ่อยแน่นอน"
 - ขั้นตอน:
 1. นับสินค้าที่ละตัว ตัวไหน support ต่ำกว่า 2% ตัดทิ้ง
 2. เอาตัวที่รอดมาจับคู่ แล้วนับใหม่ คู่ไหนต่ำกว่า 2% ตัดทิ้ง
 3. เอาคู่ที่รอดมารวมเป็นชุด 3 ชิ้น แล้วนับต่อ ทำซ้ำจนไม่มีชุดไหนรอด
 - ทำไมใช้ $\text{min_support} = 2\%$: ใบเสร็จที่มีคู่นั้นต้องมีอย่างน้อยประมาณ 360 ใบ ถ้าน้อยกว่านี้ ร้านจัดโปรไปก็ไม่คุ้ม
 - ผล: ได้ชุดสินค้าที่ขายบ่อย 399 ชุด
 - จุดอ่อน: ต้องอ่านข้อมูลซ้ำๆ รอบ และต้องสร้างตัวเลือกจำนวนมาก
- ประโยคส่งต่อ: "เลยมีอีกวิธีที่ไม่ต้องสร้างตัวเลือกเลย"

สไลด์ 7: FP-Growth และการเปรียบเทียบ

- ภาพ: กราฟ 07 (Apriori vs FP-Growth) + ภาพต้นไม้ FP-tree แบบง่าย

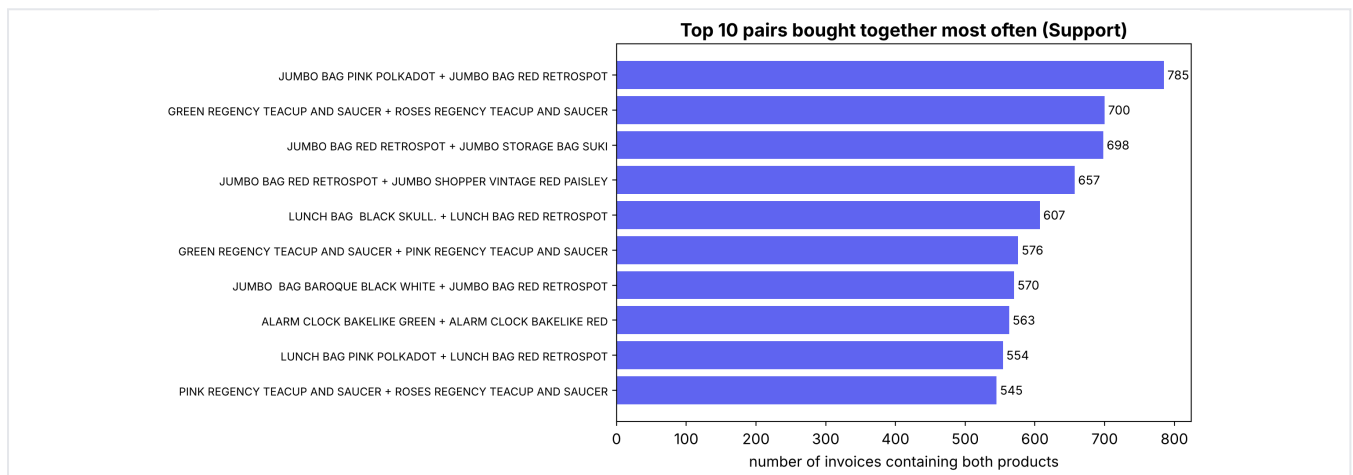


- ต้องพูด:
 - FP-Growth ทำงานยังไง:

1. อ่านข้อมูลแค่ 2 รอบ แล้วบีบใบเสร็จทั้งหมดเป็นต้นไม้ (FP-tree)
 2. ใบเสร็จที่ขึ้นต้นด้วยสินค้าเหมือนกันจะใช้กิ่งเดียวกัน
 3. หาชุดสินค้าจากต้นไม้ได้เลย ไม่ต้องสร้างตัวเลือก
- ผล: ได้ 399 ชุดเท่ากับ Apriori เป๊ะ เพราะทั้งสองวิธีหาคำตอบที่ถูกต้องครบถ้วนเหมือนกัน ต่างกันแค่ความเร็ว
 - ความเร็วใน Colab: Apriori 26.2 วินาที, FP-Growth 18.9 วินาที
 - สรุป: ถ้าข้อมูลใหญ่ขึ้น ควรเลือก FP-Growth
- ประโยคส่งต่อ: "ตอนนี้เรารู้แล้วว่าชุดไหนขายบ่อย แต่ขายบ่อยไม่ได้แปลว่าเกี่ยวกัน"

สไลด์ 8: เปลี่ยนชุดสินค้าเป็นกฎ และวิธีอ่านค่า

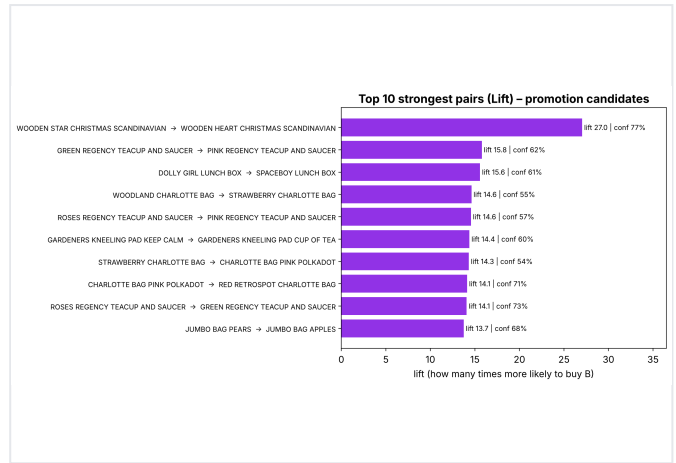
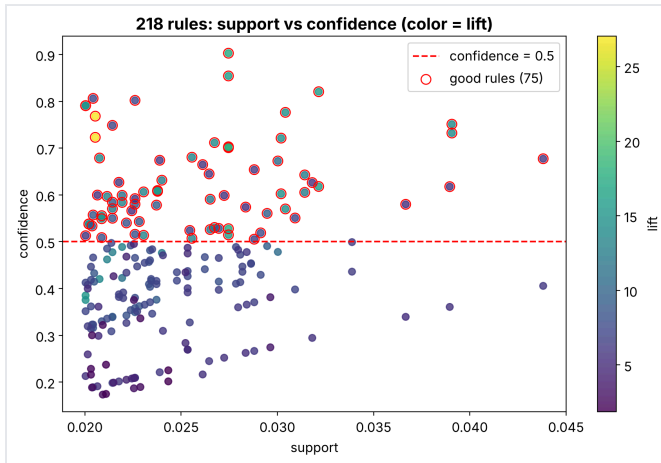
- ภาพ: ตารางตัวอย่างจริง 1 กฎ + กราฟ 05 (คู่ที่ขายบ่อยที่สุด)



- ต้องพูด: ใช้ตัวอย่างจริงคือ Jumbo Bag Pink Polkadot → Jumbo Bag Red Retrospot
 - Support = 4.4%: มี 785 จาก 17,917 ใบเสร็จที่ซื้อทั้งคู่ บอกว่าคู่นี้เกิดบ่อยแค่ไหน
 - Confidence = 68%: คนที่ซื้อถุงชมพู 68% ซื้อถุงแดงด้วย
 - Lift = 6.3: ปกติมีใบเสร็จแค่ 10.8% ที่ซื้อถุงแดง แต่ถ้าซื้อถุงชมพูแล้ว โอกาสขึ้นเป็น 68% คือมากกว่าปกติ 6 เท่า
 - ทำไมดู Lift เป็นหลัก: ยกตัวอย่างถุงพลาสติกที่ทุกใบเสร็จมี กฎ "ซื้ออะไรก็ได้ แล้วซื้อถุง" จะมี Confidence สูงทั้งที่ไม่ได้เกี่ยวกันเลย แต่ Lift จะออกมาประมาณ 1 เลยจับได้
- ประโยคส่งต่อ: "เราตั้งเกณฑ์เพื่อเก็บไว้แค่กฎที่แรงจริง"

สไลด์ 9: คัดกฎที่ดี

- ภาพ: กราฟ 08 (scatter ของกฎ) + กราฟ 06 (คู่ที่ lift สูงสุด)



- **ต้องพูด:**
 - เกณฑ์: Lift ≥ 3 และ Confidence ≥ 0.5 → จาก 218 กฎ เหลือ **75 กฎที่ดี**
 - **ชี้ในกราฟ scatter:** จุดมุมขวบนคือกฎที่ทั้งแม่นยำและแรง
 - กฎที่แรงที่สุด: Wooden Star → Wooden Heart (ของตกแต่งคริสต์มาส) Lift 27, Confidence 77%
 - **Insight:** กฎแรงๆ ส่วนใหญ่เป็นสินค้า "รุ่นเดียวกันแต่ต่างสี" เช่น แก้ว Regency ชมพู/เขียว แปลว่าลูกค้าชอบซื้อสะสมให้ครบชุด
- **ประโยคส่งต่อ:** "แต่วิธีนี้มีจุดบอดครับ สินค้าที่ขายไม่ถึง 2% จะไม่มีกฎเลย"

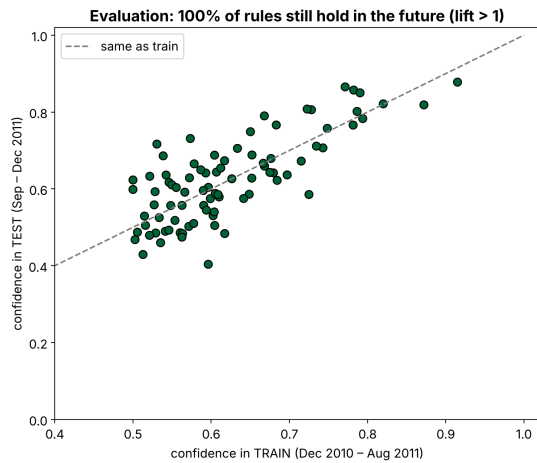
สไลด์ 10: Item2Vec เติมจุดบอด

- **ภาพ:** แผนภาพ "ใบเสร็จ = ประโยค, สินค้า = คำ"
- **ต้องพูด:**
 - **ไอเดีย:** ยืม Word2Vec จากงานภาษามาใช้ ในภาษา คำที่อยู่ในประโยคเดียวกันบ่อยๆ มักมีความหมายใกล้เคียงกัน สินค้าที่อยู่ในใบเสร็จเดียวกันบ่อยๆ ก็มักไปด้วยกัน
 - **วิธีทำ:** โมเดลแปลงสินค้าแต่ละตัวเป็นเวกเตอร์ (ชุดตัวเลข) สินค้าที่ไปด้วยกันจะมีเวกเตอร์ใกล้เคียงกัน วัดด้วย cosine similarity
 - **ผล:** ได้เวกเตอร์ครบกว่า 3,400 สินค้า รวมถึงตัวที่ Apriori เข้าไม่ถึง
 - **ตัวอย่าง:** ของที่ใกล้ White Hanging Heart T-Light Holder ที่สุดคือของแต่งบ้านธีมหัวใจ เช่น Cream Sweetheart Mini Chest, Doorstop Retrospot Heart (รายชื่อเปลี่ยนได้เล็กน้อยทุกครั้งที่ร้านใช้ตามรอบที่ Seven รันเอง)
 - **เทียบสองวิธี:** กฎ (Apriori/FP-Growth) อธิบายเหตุผลได้ชัด ส่วน Item2Vec ครอบคลุมสินค้ามากกว่า เราเลยใช้คู่กัน
- **ประโยคส่งต่อ:** "ก่อนเอาไปใช้ เราต้องพิสูจน์ก่อนว่ากฎพวกนี้เชื่อได้"

องค์ 3: พิสูจน์

สไลด์ 11: Evaluation แบ่งข้อมูลตามเวลา

- **ภาพ:** กราฟ 10 (confidence ช่วง train vs test)

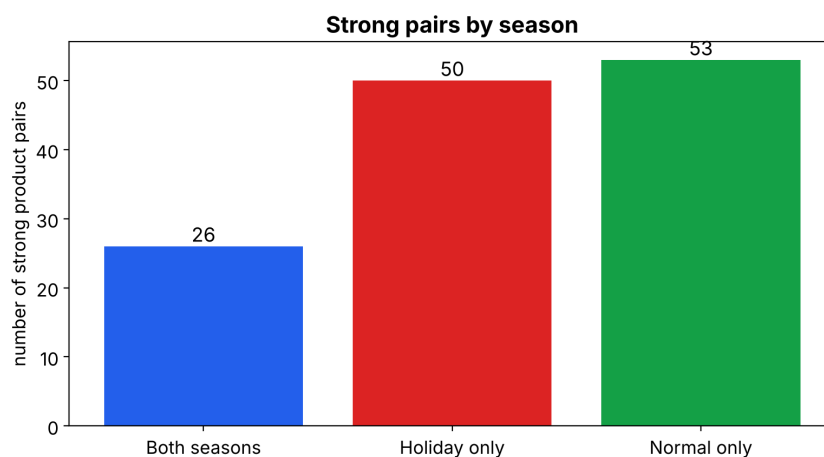


• **ต้องพูด:**

- ปัญหา: ถ้าหากกฎแล้ววัดผลบนข้อมูลชุดเดิม ก็เหมือนสอบด้วยข้อสอบที่เห็นเฉลยแล้ว
- วิธีแบ่ง:
 - Train: ธ.ค. 2010 – ส.ค. 2011 (11,264 ใบเสร็จ) ใช้หากกฎ
 - Test: ก.ย. – ธ.ค. 2011 (6,653 ใบเสร็จ) ใช้วัดผล
- ทำไมไม่สุ่มแบ่ง: ร้านจริงต้องใช้กฎจากอดีตไปทำนายอนาคต
- ผล:
 - 100% ของ 85 กฎยังมี lift > 1
 - 85% ยังผ่านเกณฑ์เดิม
 - confidence เฉลี่ยเท่าเดิมที่ 0.62
- วิธีอ่านกราฟ: จุดเกาะเส้นทแยง แปลว่ากฎไม่ได้เกิดจากความบังเอิญ

สไลด์ 12: กฎเปลี่ยนตามฤดูกาล

- ภาพ: กราฟ 11 (จำนวนคู่สินค้าแยกตามฤดูกาล) + ภาพ GPT prompt 7



• **ต้องพูด:**

- ย้อนกลับไปที่ยอดพุ่งช่วงคริสต์มาสในสไลด์ 3
- หากกฎแยกสองช่วง ได้คู่สินค้าแบบนี้
 - เจอทั้งปี 26 คู่
 - เจอเฉพาะช่วงเทศกาล 50 คู่ (ของตกแต่ยังไม่ คริสต์มาส)
 - เจอเฉพาะช่วงปกติ 53 คู่
- ข้อสรุป: ร้านไม่ควรใช้โปรชุดเดียวทั้งปี

- **วิธีทำ:** เอากราฟที่ตีมาวาดเป็นกราฟเครือข่าย (สินค้า = จุด, กฎ = เส้น) แล้วใช้อัลกอริทึม community detection (greedy modularity) แบ่งกลุ่มอัตโนมัติ
- **ผล:** ได้ 15 โชน เช่น โชน Jumbo Bag, โชน Lunch Bag, โชนแก้ว Regency, โชนคริสต์มาส
- **วิธีใช้:** ร้านหน้าร้านวางสินค้าโชนเดียวกันไว้ชั้นเดียวกัน ร้านออนไลน์เอาไปขึ้นกล่อง "Frequently bought together"

สไลด์ 15: Output #3 ระบบแนะนำสินค้า (เดโม)

- ภาพ: เปิด Colab สด หรือแคปหน้าจอฟังก์ชัน `recommend()`
- **ต้องพูด:**
 - ใส่ Jumbo Bag Red Retrospot กับ Lunch Bag Red Retrospot ลงตะกร้า ระบบแนะนำ Jumbo Bag Pink Polkadot, Jumbo Storage Bag Suki ฯลฯ
 - ลำดับการแนะนำ: ใช้กฎก่อน เพราะอธิบายได้ว่าแนะนำเพราะอะไร ถ้ากฎไม่พอค่อยใช้ Item2Vec เติม

สไลด์ 16: ข้อจำกัด ต่อยอด และปิด

- ภาพ: ภาพ GPT prompt 8
- **ต้องพูด:**
 - **ข้อจำกัด:**
 - กฎบอกได้แค่ว่าซื้อด้วยกัน (correlation) ไม่ได้พิสูจน์ว่าจัดโปรแล้วยอดขายจะเพิ่ม ถ้าจะรู้จริงต้องทำ A/B test
 - ข้อมูลมาจากร้านเดียว ปีเดียว
 - **ต่อยอด:** แนะนำ "สำหรับคุณ" รายคนด้วย Collaborative Filtering, แบ่งกลุ่มลูกค้าด้วย K-Means, เว็บไซต์สำหรับร้านไทย
 - **ประโยคปิด:** "จากใบเสร็จกว่า 5 แสนแถว เราได้กฎที่เชื่อได้ 75 ข้อ แบ่งได้ 15 โชน แยกโปรตามฤดูกาล และได้ระบบแนะนำสินค้าที่ร้านเอาไปใช้ได้จริง ขอขอบคุณครับ"

เช็กลิสต์ก่อนทำสไลด์

- ☐ สไลด์ละ 1 ประเด็น ตัวหนังสือบนสไลด์ไม่เกิน 3 บรรทัด ที่เหลือพูดเอา
- ☐ สไลด์ 5-10 (อัลกอริทึม) ใช้แผนภาพมากกว่าตัวหนังสือ
- ☐ ตัวเลขบนสไลด์ต้องตรงกับ Colab ถ้ารันใหม่แล้วตัวเลขเปลี่ยน ให้แก้ตาม
- ☐ เตรียมเปิด Colab ไว้สำหรับเดโมในสไลด์ 15 เพื่ออาจารย์ขอโค้ด

ภาคผนวก: ที่มาของตัวเลขทุกตัวในสไลด์

ใช้คู่กับ `storyline.md` ไฟล์นี้อธิบายตัวเลขทุกตัวว่า มาจากไหน, คำนวณยังไง, แปลว่าอะไร และทำไมเลือกค่านี้ เอาไว้ตอบอาจารย์ตอนถูกถามว่า "ตัวเลขนี้มาจากไหน"

ตัวเลขทุกตัวในไฟล์ได้จากการรัน notebook จริง ยกเว้น Item2Vec ที่ผลอาจต่างกันเล็กน้อยในแต่ละรอบ (ดูข้อ 7)

1. ข้อมูลดิบ (สไลด์ 3)

ค่า	มาจากไหน	แปลว่า
541,909 แถว	<code>df.shape</code> หลังโหลดไฟล์	1 แถว = สินค้า 1 รายการในใบเสร็จ ใบเสร็จ 1 ใบจึงมีได้หลายแถว
8 คอลัมน์	InvoiceNo, StockCode, Description, Quantity, InvoiceDate, UnitPrice, CustomerID, Country	เลขใบเสร็จ, รหัสสินค้า, ชื่อสินค้า, จำนวน, วันเวลา, ราคาต่อชิ้น, รหัสลูกค้า, ประเทศ
ธ.ค. 2010 – ธ.ค. 2011	ค่าต่ำสุดกับสูงสุดของ InvoiceDate	ข้อมูล 1 ปีเต็ม เลย์เห็นฤดูกาลครบ 1 รอบ

พูดว่า: "ข้อมูลนี้เป็นใบเสร็จจริงของร้านขายของชำของอังกฤษ 1 แถวคือสินค้า 1 รายการ ไม่ใช่ใบเสร็จ 1 ใบ"

2. ทำความสะอาดข้อมูล (สไลด์ 4)

ปัญหาที่เจอ	จำนวน	วิธีแก้	ทำไมต้องตัด
ใบเสร็จยกเล็ก	9,288	InvoiceNo ขึ้นต้นด้วย "C"	ลูกค้าไม่ได้ซื้อจริง ถ้าเก็บไว้จะได้คู่สินค้าปลอม
<code>Quantity ≤ 0</code>	10,624	<code>Quantity <= 0</code>	จำนวนติดลบคือการคืนของหรือปรับสต็อก ไม่ใช่การขาย
<code>UnitPrice ≤ 0</code>	2,517	<code>UnitPrice <= 0</code>	ราคา 0 คือของแถมหรือข้อมูลผิด
ไม่มีชื่อสินค้า	1,454	Description เป็นค่าว่าง	ไม่รู้ว่าเป็นสินค้าอะไร เลยจับคู่ไม่ได้
ค่าส่ง ค่าขนส่ง	–	POSTAGE, DOTCOM POSTAGE, CARRIAGE, Manual	ไม่ใช่สินค้า ถ้าเก็บไว้ ระบบจะบอกว่า "ทุกอย่างขายคู่กับค่าส่ง"
ไม่มี CustomerID	135,080	CustomerID เป็นค่าว่าง	ไม่ได้ตัด เพราะงานนี้ดูแค่ใบเสร็จ ไม่ได้ดูว่าลูกค้าเป็นใคร

ผลหลังทำความสะอาด: 541,909 → 527,809 แถว (ตัดไปประมาณ 2.6%)

แต่ละปัญหาซ้อนกันได้ เช่น ใบเสร็จยกเล็กก็มี Quantity ติดลบด้วย จำนวนที่ตัดจริงเลยน้อยกว่าผลรวมของทุกแถวในตาราง

ถ้าอาจารย์ถามว่าทำไมไม่ตัดแถวที่ไม่มี CustomerID: "เพราะเราหาคู่สินค้าจากใบเสร็จ แถวที่ไม่มีรหัสลูกค้ายังมีเลขใบเสร็จครบ ถ้าตัดทิ้งจะเสียข้อมูลไปถึง 25%"

3. แปลงเป็นตะกร้า (สไลด์ 4)

ค่า	มาจากไหน	แปลว่า
United Kingdom	COUNTRY = "United Kingdom"	เลือกอังกฤษเพราะเป็นข้อมูลประมาณ 90% ของทั้งหมด ถ้ารวมทุกประเทศ พฤติกรรมลูกค้าคนละแบบจะปนกัน
17,917 ใบเสร็จ	จำนวน InvoiceNo ที่ไม่ซ้ำกัน ของลูกค้าในอังกฤษ	คือจำนวน "ตะกร้า" ที่ใช้หากฎ
3,992 สินค้า	จำนวน Description ที่ไม่ซ้ำกัน	คือจำนวนคอลัมน์ในตาราง basket
26 ชั้นต่อใบเสร็จ (เฉลี่ย)	ค่าเฉลี่ยของจำนวนสินค้าในแต่ละใบเสร็จ (ค่ากลาง 15 ชั้น)	ร้านนี้ขายส่งด้วย ลูกค้าหลายคนเป็นร้านค้าที่ซื้อทีละเยอะๆ ตะกร้าเลยใหญ่กว่าร้านปลีกทั่วไป

TransactionEncoder ทำอะไร: แปลงใบเสร็จ 1 ใบให้เป็นแถว True/False 3,992 ช่อง ช่องไหนเป็น True แปลว่าใบเสร็จนั้นมีสินค้าตัวนั้น เราไม่สนว่าซื้อกี่ชิ้น สนแค่ว่ามีหรือไม่มี

4. ทำมันับตรงๆ ไม่ได้ (สไลด์ 5)

ค่า	วิธีคิด	ผล
คู่สินค้าที่เป็นไปได้	เลือก 2 จาก 3,992 = $3,992 \times 3,991 \div 2$	ประมาณ 8 ล้านคู่
ชุด 3 ชั้นที่เป็นไปได้	เลือก 3 จาก 3,992 = $3,992 \times 3,991 \times 3,990 \div 6$	ประมาณ 1 หมื่นล้านชุด

พูดว่า: "ถ้าจะนับทุกชุดในทุกใบเสร็จ ต้องนับเป็นหมื่นล้านครั้งคุณ 18,000 ใบเสร็จ คอมพิวเตอร์เราเลยต้องมีวิธีตัดตัวเลือกทิ้งก่อน"

5. Apriori และ min_support (สไลด์ 6)

Support คืออะไร

สูตร: $\text{Support}(X) = \text{จำนวนใบเสร็จที่มี } X \div \text{จำนวนใบเสร็จทั้งหมด}$

ตัวอย่าง: White Hanging Heart T-Light Holder อยู่ใน 12.1% ของใบเสร็จ ซึ่งเป็นสินค้าที่ขายบ่อยที่สุด

ทำไมเลือก min_support = 0.02 (2%)

- 2% ของ 17,917 ใบเสร็จ $\approx$ 358 ใบเสร็จ แปลว่าชุดสินค้าต้องถูกซื้ออย่างน้อยประมาณ 358 ครั้งต่อปี ร้านถึงจะคุ้มที่จะจัดโปร
- ถ้าตั้งต่ำกว่านี้ (เช่น 1%): ได้ชุดสินค้าเยอะขึ้นมาก รันช้าลง และได้กฎของสินค้าที่ขายนานๆ ครั้ง ซึ่งเอาไปทำโปรได้ไม่คุ้ม
- ถ้าตั้งสูงกว่านี้ (เช่น 5%): แทบไม่เหลือคู่สินค้าเลย เพราะสินค้าที่ขายดีทีสุดยังอยู่ในใบเสร็จแค่ 12%

Apriori ตัดตัวเลือกได้เยอะแค่ไหน (ตัวเลขจริง)

รอบ	ทำอะไร	ผ่าน min_support
1	นับสินค้าที่ละตัว ทั้งหมด 3,992 ตัว	300 ตัว (ตัดทิ้งไป 3,692 ตัว)
2	จับคู่เฉพาะ 300 ตัวที่รอด = $300 \times 299 \div 2 \approx 44,850$ คู่ (แทนที่จะเป็น 8 ล้าน)	94 คู่
3	รวมคู่ที่รอดเป็นชุด 3 ชั้น	5 ชุด
4	รวมเป็นชุด 4 ชั้น	ไม่มีชุดไหนรอด หยุด

รวม $300 + 94 + 5 = 399$ itemsets

พูดว่า: "หลัก Apriori บอกว่าถ้าสินค้าตัวเดียวขายไม่ถึง 2% คู่ที่มีมันก็ไม่มีทางถึง 2% แคร่รอบแรกก็ตัดสินค้าทิ้งไปกว่า 90% ทำให้คู่ที่ต้องนับลดจาก 8 ล้านเหลือ 4.5 หมื่น"

6. FP-Growth (สไลด์ 7)

ค่า	มาจากไหน	แปลว่า
399 itemsets เท่ากัน	notebook เช็คให้ว่า <code>set(itemsets_apriori) == set(itemsets_fp)</code> ได้ True	ทั้งสองวิธีหาคำตอบที่ถูกต้องครบทุกชุด ต่างกันแค่วิธีหา
Apriori 26.2 วินาที, FP-Growth 18.9 วินาที	จับเวลาด้วย <code>time.time()</code> ใน Colab ของ Seven	FP-Growth เร็วกว่าประมาณ 30%

ทำไม FP-Growth เร็วกว่า:

- Apriori อ่านข้อมูลใหม่ทุกรอบ และต้องสร้างคู่ตัวเลือก 44,850 คู่ก่อนนับ
- FP-Growth อ่านข้อมูลแค่ 2 รอบ
 - รอบแรก: นับสินค้าที่ละตัว
 - รอบสอง: บีบใบเสร็จทั้งหมดเป็นต้นไม้ โดยใบเสร็จที่มีสินค้าขึ้นต้นเหมือนกันจะใช้กิ่งเดียวกัน
- จากนั้นหาชุดสินค้าจากต้นไม้ได้เลย ไม่ต้องสร้างตัวเลือก

ถ้าอาจารย์ถามว่าทำไมตัวเลขเวลาไม่ตรงกับในไฟล์: เวลาขึ้นกับเครื่องที่รัน แต่ละรอบจะไม่เท่ากัน สิ่งที่สำคัญคือ FP-Growth เร็วกว่าทุกครั้ง

7. จากชุดสินค้าเป็นกฎ และการอ่าน 3 ค่า (สไลด์ 8)

218 กฎมาจากไหน

ชุดสินค้า 1 ชุดแตกเป็นกฎได้หลายทิศ

- คู่ 2 ชั้น {A, B}: แยกได้ 2 กฎ คือ $A \rightarrow B$ และ $B \rightarrow A$
 - $94 \text{ คู่} \times 2 = 188 \text{ กฎ}$
- ชุด 3 ชั้น {A, B, C}: แยกได้ 6 กฎ เช่น $A \rightarrow BC$, $AB \rightarrow C$
 - $5 \text{ ชุด} \times 6 = 30 \text{ กฎ}$
- รวม $188 + 30 = 218 \text{ กฎ}$ (ทุกกฎมี lift > 1 จึงผ่าน `min_threshold=1` ครบ)

ตัวอย่างคำนวณด้วยมือ: Jumbo Bag Pink Polkadot → Jumbo Bag Red Retrosport

ตัวเลขตั้งต้นจากข้อมูล

- ใบเสร็จทั้งหมด = 17,917
- มีถุงชมพู (A) = 1,159 ใบ
- มีถุงแดง (B) = 1,935 ใบ
- มีทั้งคู่ = 785 ใบ

ค่า	สูตร	คำนวณ	ผล	แปลว่า
Support	ใบเสร็จที่มีทั้ง A และ B ÷ ใบเสร็จทั้งหมด	$785 \div 17,917$	4.4%	คุณเกิดบ่อยแค่ไหน (เกิน 2% จึงผ่าน)
Confidence	ใบเสร็จที่มีทั้ง A และ B ÷ ใบเสร็จที่มี A	$785 \div 1,159$	68%	ซื้อถุงชมพูแล้ว 68% ซื้อถุงแดงด้วย
ความนิยมของ B	ใบเสร็จที่มี B ÷ ใบเสร็จทั้งหมด	$1,935 \div 17,917$	10.8%	ปกติคนซื้อถุงแดงแค่ 10.8%
Lift	Confidence ÷ ความนิยมของ B	$68\% \div 10.8\%$	6.3	ซื้อถุงชมพูแล้ว โอกาสซื้อถุงแดงสูงกว่าปกติ 6.3 เท่า

วิธีอ่าน Lift

- Lift = 1: ไม่เกี่ยวกันเลย ซื้อ A หรือไม่ซื้อ โอกาสซื้อ B ก็เท่าเดิม
- Lift > 1: ซื้อด้วยกันบ่อยกว่าที่ควรเป็น
- Lift < 1: ซื้อ A แล้วกลับไม่ค่อยซื้อ B

ทำไมดู Lift ก่อน → Confidence → Support

- Lift: บอกว่า "เกี่ยวกันจริงไหม" โดยหักความนิยมของ B ออกไปแล้ว ถ้าไม่เกี่ยวกัน ค่าอื่นจะสูงแค่ไหนก็ไม่มี ความหมาย
- Confidence: บอกว่า "ถ้าจัดโปร จะได้ผลบ่อยแค่ไหน"
- Support: บอกว่า "คุ้มไหมที่จะจัดโปร" เราใช้กรองไว้ตั้งแต่ขั้น Apriori แล้ว

8. คัดกฎที่ดี (สไลด์ 9)

เกณฑ์	ทำไมเลือกค่านี้
Lift ≥ 3	อยากได้แค่กฎที่การซื้อ A ทำให้โอกาสซื้อ B เพิ่มขึ้นอย่างน้อย 3 เท่า ถ้าน้อยกว่านี้ ความสัมพันธ์อ่อนเกินไปที่จะจัดโปร
Confidence ≥ 0.5	ลูกค้าที่ซื้อ A อย่างน้อยครึ่งหนึ่งต้องซื้อ B ด้วย ถ้าน้อยกว่าครึ่ง การแนะนำจะผิดพลาดกว่าถูก

ผล: 218 → 75 กฎที่ดี ในนั้นเป็นกฎแบบ 1 ต่อ 1 (ซื้อ 1 อย่าง แนะนำ 1 อย่าง) 58 กฎ ซึ่งเป็นกลุ่มที่เราเอาไปทำโปรกับจัดโซน

กฎที่แรงที่สุด: Wooden Star → Wooden Heart (Christmas Scandinavian)

- Support 2.05%: แทบจะเฉียดเกณฑ์ 2%
- Confidence 77%
- Lift 27: ซื้อดาวไม้แล้ว โอกาสซื้อหัวใจไม้สูงกว่าปกติ 27 เท่า

ถ้าอาจารย์ถามว่าเกณฑ์ 3 กับ 0.5 มาจากไหน: "เป็นเกณฑ์ที่เราตั้งเองตามโจทย์ธุรกิจ ไม่ใช่ค่ามาตรฐาน ถ้าร้านอยากได้กฎเยอะขึ้นก็ลดเกณฑ์ได้ แต่จะได้กฎที่อ่อนลง"

9. Item2Vec (สไลด์ 10)

ค่าที่ตั้ง	ความหมาย	ทำไมเลือก
vector_size=50	สินค้า 1 ตัวแทนด้วยตัวเลข 50 ตัว	พอสำหรับสินค้าประมาณ 4,000 ตัว ถ้าใช้เยอะเกินจะต้องใช้ข้อมูลมากกว่านี้
window=10	ดูสินค้าที่อยู่ใกล้กันในใบเสร็จได้ไกล 10 ตัว	ในใบเสร็จไม่มีลำดับเหมือนประโยค เลยตั้งกว้างไว้ให้เห็นสินค้าทั้งตะกร้า
min_count=5	สินค้าต้องอยู่ในอย่างน้อย 5 ใบเสร็จ	ถ้าน้อยกว่านี้ โมเดลเรียนรู้สินค้าตัวนั้นไม่ได้
sg=1	ใช้วิธี Skip-gram	ใช้สินค้า 1 ตัวทายสินค้ารอบๆ ทำงานกับของที่ขายไม่บ่อยได้ดีกว่าอีกริธี (CBOW)
epochs=10	อ่านข้อมูลทั้งหมด 10 รอบ	ให้โมเดลเรียนรู้ซ้ำจนนิ่ง

ผล: ได้เวกเตอร์ 3,492 สินค้า เทียบกับ Apriori ที่มีกฎครอบคลุมสินค้าแค่หลักสิบตัว

Similarity คืออะไร: cosine similarity ระหว่างเวกเตอร์ 2 ตัว มีค่า -1 ถึง 1 ยิ่งใกล้ 1 ยิ่งมักอยู่ในตะกร้าแบบเดียวกัน

ตัวอย่างจากการรัน: สินค้าที่ใกล้ White Hanging Heart T-Light Holder ที่สุดคือ

- Cream Sweetheart Mini Chest (0.79)
- Doorstop Retrospot Heart (0.77)
- Filigree Heart Bird White (0.76)

ทั้งหมดเป็นของแต่งบ้านธีมหัวใจ

ข้อควรรู้: Word2Vec สุ่มค่าเริ่มต้นและรันหลายเรดพร้อมกัน รายชื่อสินค้ากับค่า similarity เลยอาจเปลี่ยนเล็กน้อยทุกครั้งที่รัน ก่อนทำสไลด์ให้ใช้ตัวเลขจากรอบที่ Seven รันเอง

10. Evaluation (สไลด์ 11)

ค่า	มาจากไหน	แปลว่า
SPLIT_DATE = 1 ก.ย. 2011	ตัวเอง	ใช้ข้อมูลประมาณ 9 เดือนแรกหากฎ และเก็บ 3 เดือนครึ่งสุดท้ายไว้สอบ
Train 11,264 / Test 6,653 ใบเสร็จ	นับใบเสร็จก่อนและหลังวันแบ่ง	ประมาณ 63 : 37 ฝั่ง test ใหญ่กว่าปกติเพราะช่วงท้ายปีขายดี
85 กฎที่ดีจาก train	หากฎด้วยเกณฑ์เดิม โดยใช้แค่ข้อมูล train	ได้มากกว่า 75 กฎของทั้งปี เพราะช่วงนี้ไม่มีของคริสต์มาสมาปน
100% มี lift > 1 ใน test	เอากฎทั้ง 85 ข้อไปคำนวณ support, confidence และ lift ใหม่บนใบเสร็จ test	ทุกกฎยังเป็นความสัมพันธ์จริง ไม่มีกฎไหนที่เกิดจากความบังเอิญ
85% ผ่านเกณฑ์เดิม	นับกฎที่อยู่ใน test ยังมี lift ≥ 3 และ confidence ≥ 0.5	มีกฎ 15% ที่อ่อนลงบ้าง แต่ยังเกี่ยวกันอยู่
Confidence เฉลี่ย 0.62 $\rightarrow$ 0.62	ค่าเฉลี่ยของทั้ง 85 กฎ ใน train เทียบกับใน test	ความแม่นยำไม่ลดลงเลย
Lift เฉลี่ย 9.7 $\rightarrow$ 11.9	ค่าเฉลี่ยเหมือนกัน	lift สูงขึ้นเพราะช่วงเทศกาลมีสินค้าหลากหลายชิ้น ทำให้ความนิยมของสินค้าแต่ละตัวต่ำลง lift (ซึ่งหารด้วยความนิยม) จึงสูงขึ้น

ทำไมแบ่งตามเวลา ไม่สุ่ม:

- ถ้าสุ่ม ใบเสร็จวันเดียวกันจะไปอยู่ทั้ง train และ test ผลจะดูดีเกินจริง (เรียกว่า data leakage)
- ร้านจริงต้องใช้กฎจากอดีตไปทำนายอนาคต การแบ่งตามเวลาจึงตรงกับการใช้งานจริง

11. ฤดูกาล (สไลด์ 12)

ค่า	มาจากไหน
ช่วงเทศกาล = เดือน 9–12, ช่วงปกติ = เดือน 1–8	ดูจากกราฟยอดขายรายเดือน ยอดเริ่มพุ่งตั้งแต่ ก.ย.
26 / 50 / 53 คู่	หากฎแยกในแต่ละช่วงด้วยเกณฑ์เดิม รวม $A \rightarrow B$ กับ $B \rightarrow A$ เป็นคู่เดียว แล้วเทียบว่าคู่ไหนเจอในช่วงไหน

ตีความ

- เจอทั้งปี 26 คู่: เป็นคู่ที่แข็งแกร่งจริง ใช้เป็นโปรประจำได้
- เจอเฉพาะช่วงเทศกาล 50 คู่: ส่วนใหญ่เป็นของตกแต่งคริสต์มาสไม้ เช่น Wooden Stocking + Wooden Sleigh (lift 23.7)
- เจอเฉพาะช่วงปกติ 53 คู่: ตอนเข้าเทศกาลลูกค้าเปลี่ยนไปซื้ออย่างอื่น คู่พวกนี้เลยหายไป

ข้อควรระวัง: ช่วงปกติมี 8 เดือน ส่วนช่วงเทศกาลมีแค่ 4 เดือน แต่เกณฑ์ support คิดเป็น % จึงยังเทียบกันได้

12. รายได้จากโปร (สไลด์ 13)

สูตร: รายได้เพิ่ม = (ใบเสร็จที่มี A แต่ไม่มี B) × CONVERSION × ราคา B × (1 - DISCOUNT)

ตัวอย่างคำนวณ: ช้อนนาฬิกาปลูก Bakelike สีแดง ลดสีเขียว

ส่วนของสูตร	ค่า	มาจากไหน
ใบเสร็จที่มี A แต่ไม่มี B	367	นับใบเสร็จที่มีสีแดงแต่ไม่มีสีเขียว = ลูกค้าที่โปรนี้ดึงมาได้
CONVERSION	10%	สมมติฐาน: ลูกค้า 1 ใน 10 ที่เห็นโปรแล้วซื้อเพิ่ม
ราคา B	£3.75	ค่ากลาง (median) ของราคานาฬิกาปลูกสีเขียวในข้อมูล ใช้ median เพราะไม่โดนราคาผิดปกติ
DISCOUNT	10%	สมมติฐาน: ลดราคา B ให้ 10%
รวม	$367 \times 0.10 \times 3.75 \times 0.90 =$ £124 ต่อปี	

รวม 10 โปรแรก ≈ £976 ต่อปี

ทำไมโปรอันดับ 1 ไม่ใช่กฎที่ lift สูงสุด: รายได้ขึ้นกับจำนวนลูกค้าที่ยังไม่ซื้อ B กับราคา B ไม่ใช่ lift

- กฎที่ confidence สูงมาก แปลว่าลูกค้าส่วนใหญ่ซื้อคู่กันอยู่แล้ว เหลือลูกค้าให้ดึงน้อย
- ลดราคาให้คนที่ตั้งใจซื้ออยู่แล้ว ร้านก็เสียกำไรฟรี

ถ้าอาจารย์ถามว่าตัวเลขเชื่อได้ไหม: "เป็นค่าประมาณจากสมมติฐาน 10% ครับ ใช้เทียบว่าโปรไหนคุ้มกว่ากัน ไม่ได้ใช้ทำนายรายได้จริง ถ้าจะรู้จริงต้องทดลองแบบ A/B test"

13. จัดโซนสินค้า (สไลด์ 14)

ค่า	มาจากไหน
กราฟเครือข่าย	เอากฎ 1 ต่อ 1 ทั้ง 58 กฎมาวาด สินค้า = จุด, กฎ = เส้นเชื่อม
Greedy modularity	อัลกอริทึมหากลุ่ม (community detection) เริ่มจากให้ทุกจุดเป็นกลุ่มของตัวเอง แล้วรวมกลุ่มที่ทำให้ค่า modularity สูงขึ้นมากที่สุดทีละคู่ จนรวมต่อไม่ได้แล้ว
Modularity คืออะไร	วัดว่าเส้นเชื่อมอยู่ "ในกลุ่ม" มากกว่าที่ควรเป็นแค่ไหน ยิ่งสูง ยิ่งแปลว่าในกลุ่มเชื่อมกันแน่น แต่ข้ามกลุ่มเชื่อมกันน้อย
15 โซน	ผลที่อัลกอริทึมแบ่งได้เอง เราไม่ได้กำหนดจำนวนโซน (ต่างจาก K-Means ที่ต้องบอกจำนวนกลุ่มก่อน)

ตัวอย่างโซน: Jumbo Bag 5 ตัว, Charlotte Bag 5 ตัว, Lunch Bag 5 ตัว, แก้ว Regency 3 สี, นาฬิกาปลูก Bakelike 3 สี

14. ระบบแนะนำ recommend() (สไลด์ 15)

ทำงานยังไง

1. รับสินค้าในตะกร้าเข้ามา
2. หากกฎที่ฝั่ง "ซื้อ" มีสินค้าอยู่ในตะกร้าครบ
3. เรียงตาม lift จากมากไปน้อย แล้วเอาฝั่ง "แนะนำ" ที่ยังไม่อยู่ในตะกร้ามาแสดง
4. ถ้าได้ไม่ครบ k ตัว (ค่าเริ่มต้น 5) ให้เอาสินค้าที่ Item2Vec บอกว่าใกล้เคียงที่สุดมาเติม

ตัวอย่างจริง: ตะกร้ามี Jumbo Bag Red Retrosport และ Lunch Bag Red Retrosport ระบบแนะนำ

- Jumbo Bag Pink Polkadot
- Jumbo Shopper Vintage Red Paisley
- Jumbo Storage Bag Suki
- Lunch Bag Pink Polkadot
- Lunch Bag Woodland

ทำไมใช้กฎก่อน Item2Vec: กฎอธิบายได้ว่าแนะนำเพราะอะไร เช่น "68% ของคนที่ซื้อถุงชมพูซื้อถุงแดงด้วย" ส่วน Item2Vec บอกได้แค่ว่าเวกเตอร์ใกล้กัน